Object Oriented Programming (CSEG1044)
Unit 06: Capstone Project

Phase 4: UI and Integration

Project: Cafeteria Order and Billing System

Covers: Swing screens, service layer wiring, end-to-end workflow integration

1. Goal of Phase 4

Phase 4 converts the class design and DAO layer into a working desktop application. All Swing screens are built in this phase and connected to the service layer. The golden rule of Phase 4 is: UI panels call service methods only — never DAO methods directly, and never perform calculations.

2. Screen Inventory

The application has 8 screens. They are built in the order shown below — simpler screens first.

#	Java File	Type	Purpose
1	`LoginFrame.java`	`JFrame`	First screen. Authenticates user via UserService. Routes Admin to AdminDashboard, Operator to OperatorDashboard.
2	`AdminDashboard.java`	`JFrame`	Admin home screen. Three buttons: Manage Menu, Order History, Daily Sales Summary.
3	`OperatorDashboard.java`	`JFrame`	Operator home screen. Two buttons: Place New Order, Order History.
4	`MenuManagementPanel.java`	`JPanel`	Admin adds, edits, deletes menu items. Table refreshes from DB after every change.
5	`PlaceOrderPanel.java`	`JPanel`	Operator selects items, sets quantities, enters customer name, places order. Bill is auto-generated.
6	`BillPanel.java`	`JPanel`	Displays the formatted bill receipt after an order is placed. Shows itemised breakdown, tax, and total.
7	`OrderHistoryPanel.java`	`JPanel`	Shows all past orders in a table. Clicking a row reveals item-by-item breakdown below.
8	`SalesSummaryPanel.java`	`JPanel`	Admin picks a date. Shows total orders, total revenue, and individual order rows for that day.

3. Screen Navigation Flow

The diagram below shows how screens connect to each other:

```
App launches  ──▶  LoginFrame
    │
    ├── [Admin credentials]   ──▶  AdminDashboard
    │       ├── [Manage Menu]      ──▶  MenuManagementPanel (in new JFrame)
    │       ├── [Order History]    ──▶  OrderHistoryPanel   (in new JFrame)
    │       ├── [Sales Summary]    ──▶  SalesSummaryPanel   (in new JFrame)
    │       └── [Logout]           ──▶  LoginFrame
    │
    └── [Operator credentials] ──▶  OperatorDashboard
            ├── [Place Order]      ──▶  PlaceOrderPanel     (in new JFrame)
            │       └── [After placing order] ──▶  BillPanel (in new JFrame)
            ├── [Order History]    ──▶  OrderHistoryPanel   (in new JFrame)
            └── [Logout]           ──▶  LoginFrame
```

4. Integration Architecture

Every screen follows the same call chain. Data flows in one direction:

```
UI Panel (Swing)   calls ──▶   Service Class   calls ──▶   DAO Class   talks to ──▶
MySQL DB

UI Panel  ──  only handles display and user input events
Service   ──  validates input, applies business rules, orchestrates DAO calls
DAO       ──  executes SQL, maps ResultSet to model objects
Model     ──  plain Java objects passed between all layers
```

UI Screen	Service(s) Used	What the Service Does
`LoginFrame`	`UserService`	Calls UserDao.findByUsername(), then user.validatePassword()
`MenuManagementPanel`	`MenuService`	Validates name/price/category, checks uniqueness, calls MenuDao CRUD
`PlaceOrderPanel`	`OrderService, BillingService`	Validates order, calls OrderDao.saveOrder(), then generates Bill
`BillPanel`	`BillingService`	Calls formatBillText() to build the receipt string
`OrderHistoryPanel`	`OrderService`	Calls getOrderHistory() — returns List<Order> from OrderDao

SalesSummaryPanel	OrderService	Calls getDailySummary(date) and getTotalRevenue(orders)

5. Screen-by-Screen Details

5.1 LoginFrame

Property	Detail
Components	JTextField (username), JPasswordField (password), JButton (Login), JLabel (error)
Success Flow	UserService.login() returns a User → check user.isAdmin() → open correct dashboard, dispose LoginFrame
Failure Flow	UserService returns null → show 'Invalid username or password' in red JLabel, clear password field
Validation	Both fields must be non-empty before calling service. Enter key on password field triggers login.
No SQL here	All authentication SQL is in UserDao.findByUsername()

5.2 MenuManagementPanel

Property	Detail
Components	JTable (item list), JTextField (name, category, price), JCheckBox (available), JButton (Add, Update, Delete, Clear), JLabel (status)
On Load	Calls MenuService.getAllItems() and populates JTable
Row Select	Fills form fields with selected item data. Enables Update and Delete buttons, disables Add.
Add Item	Builds MenuItem from form → MenuService.addItem() → reloads table. Shows green status on success.
Update Item	Builds MenuItem with selectedItemId → MenuService.updateItem() → reloads table
Delete Item	Shows JOptionPane confirmation → MenuService.deleteItem() → reloads table
Validation	Name/category empty, price not a number, duplicate name — all caught from MenuService exception and shown in status JLabel
Table Refresh	After every add/update/delete, tableModel is cleared and reloaded from DB — never patched in memory

5.3 PlaceOrderPanel

Property	Detail
Layout	JSplitPane: left = available menu table, right = current order table + customer fields
On Load	Calls MenuService.getAvailableItems() to populate left table
Add to Order	Select item → enter quantity → click Add. Adds OrderItem to in-memory list. Subtotal label updates in real time.
Duplicate Item	If same item added twice, quantity increases — a new row is not created
Remove Item	Select row in order table → click Remove. Removes from in-memory list, subtotal updates.
Place Order	Builds Customer and Order from form → OrderService.placeOrder() → BillingService.generateBill() → opens BillPanel in new JFrame
Validation	Customer name required, at least one item required, quantity must be positive integer
After Success	Order items list is cleared, customer fields reset, status shows Bill ID

5.4 BillPanel

Property	Detail
Components	JTextArea (non-editable, monospace font), JButton (Close)
Content	Calls BillingService.formatBillText(bill) which returns plain-text receipt with item breakdown, subtotal, GST, and total
Font	Courier New 13pt — ensures receipt formatting aligns correctly
No Calculation	BillPanel only displays text — all numbers come from the Bill object passed in

5.5 OrderHistoryPanel

Property	Detail
Layout	JSplitPane vertical: top = orders table, bottom = order detail text area
Columns	Order ID, Customer, Date/Time, Items (count), Total (Rs.), Status
Row Click	ListSelectionListener fires → builds formatted item breakdown string → shows in JTextArea below

Sort Order	Most recent orders appear at the top (ORDER BY timestamp DESC in OrderDao)
Refresh	Refresh button reloads all orders from DB
Both roles	Admin and Operator can view order history — accessible from both dashboards

5.6 SalesSummaryPanel

Property	Detail
Components	JSpinner (date picker), JButton (Load Summary), stat JLabels, JTable
Date Selection	JSpinner with SpinnerDateModel and DateEditor formatted as yyyy-MM-dd
On Load	OrderService.getDailySummary(date) returns List<Order> → OrderService.getTotalRevenue() gives day total
Summary Stats	Total Orders count and Total Revenue displayed in bordered, coloured JLabel boxes
Table Columns	Order ID, Customer, Time, Items count, Total (Rs.)
No Orders	If no orders for the date: JOptionPane shows 'No sales recorded for this date'
Admin only	Only accessible from AdminDashboard

6. Integration Rules

All team members must follow these rules when writing Phase 4 code:

Rule	Reason
UI panels call service methods only, never DAO methods	Keeps SQL out of the UI layer completely
No SQL or JDBC imports in any UI class	Enforces layered architecture
No calculation logic inside ActionListeners	BillingService owns all calculation — tax rate changes in one place
Always catch exceptions in the UI and show them in a JLabel or JOptionPane	Users must never see a Java stack trace

Reload table from DB after every CUD operation	Ensures the screen reflects the real database state
Use SwingUtilities.invokeLater() to launch the first frame	Ensures Swing runs on the Event Dispatch Thread
Dispose the current frame before opening a new dashboard	Prevents multiple dashboard windows from being open

7. Bill Calculation Logic

Bill calculation lives entirely inside BillingService. The formula used:

```
Subtotal   =  sum of (unit_price × quantity) for all OrderItems
Tax        =  Subtotal × 0.05              (5% GST)
Total      =  Subtotal + Tax

Example:
  Veg Thali × 2  =  Rs. 140.00
  Samosa    × 3  =  Rs.  60.00
  ──────────────────────────
  Subtotal       =  Rs. 200.00
  GST (5%)       =  Rs.  10.00
  TOTAL          =  Rs. 210.00
```

Duplicate billing is blocked by OrderDao.isBilled(orderId) which checks if a row already exists in the bills table for that order. BillingService throws IllegalStateException if already billed.

8. Complete File List for Phase 4

File	Package	Layer
`Main.java`	`(root)`	Entry point — launches LoginFrame via SwingUtilities.invokeLater
`LoginFrame.java`	`cafeteria.ui`	UI — Presentation Layer
`AdminDashboard.java`	`cafeteria.ui`	UI — Presentation Layer
`OperatorDashboard.java`	`cafeteria.ui`	UI — Presentation Layer
`MenuManagementPanel.java`	`cafeteria.ui`	UI — Presentation Layer
`PlaceOrderPanel.java`	`cafeteria.ui`	UI — Presentation Layer
`BillPanel.java`	`cafeteria.ui`	UI — Presentation Layer
`OrderHistoryPanel.java`	`cafeteria.ui`	UI — Presentation Layer

`SalesSummaryPanel.java`	`cafeteria.ui`	UI — Presentation Layer
`UserService.java`	`cafeteria.service`	Service — Business Logic Layer
`MenuService.java`	`cafeteria.service`	Service — Business Logic Layer
`OrderService.java`	`cafeteria.service`	Service — Business Logic Layer
`BillingService.java`	`cafeteria.service`	Service — Business Logic Layer

9. End-to-End Workflow Test

Follow this exact sequence to verify the complete system works:

Step	Action	Expected Result
1	**Run schema.sql in MySQL**	Database cafeteria_db created with 6 tables and seed data
2	**Update DBConnection.java with your MySQL password**	No error needed here — verified in next step
3	**Run Main.java**	LoginFrame appears centred on screen
4	**Login as admin / admin123**	AdminDashboard opens. LoginFrame closes.
5	**Click Manage Menu Items**	MenuManagementPanel opens. Table shows 6 seed items.
6	**Add a new item: 'Lassi', Beverages, Rs. 35**	Item appears in table immediately after Save.
7	**Try adding another item named 'Lassi'**	Error shown: 'An item with the name Lassi already exists.'
8	**Logout and login as operator / operator123**	OperatorDashboard opens.
9	**Click Place New Order**	PlaceOrderPanel opens. Left table shows available items.
10	**Select Veg Thali qty 2, Samosa qty 3. Enter customer: Riya Sharma**	Order table shows both items. Subtotal: Rs. 200.00
11	**Click Place Order & Generate Bill**	BillPanel opens showing itemised bill. Total: Rs. 210.00 (with 5% GST).
12	**Close BillPanel. Click View Order History**	Order for Riya Sharma appears at top. Click it — items shown below.

13	**Logout. Login as admin. Click Daily Sales Summary**	Select today's date. Click Load Summary. Order count and revenue correct.

10. Phase 4 Review Checklist

	Checklist Item
✓	Login routes Admin → AdminDashboard and Operator → OperatorDashboard correctly
✓	Wrong password shows error in UI — no stack trace, no console print
✓	Empty username or password shows validation message before calling service
✓	MenuManagementPanel table loads all menu items on open
✓	Add Item validates: name not empty, category not empty, price > 0
✓	Duplicate item name is rejected with a clear message
✓	Table reloads from database after every add, update, or delete
✓	PlaceOrderPanel subtotal label updates in real time as items are added
✓	Duplicate menu item added to order increases quantity, not a new row
✓	Order cannot be placed with no items selected
✓	Order cannot be placed without a customer name
✓	Quantity must be a positive integer — zero or negative rejected
✓	BillPanel shows correct subtotal, 5% GST, and final total
✓	Attempting to bill the same order twice is blocked with an error
✓	OrderHistoryPanel shows most recent orders at the top
✓	Clicking an order row shows complete item breakdown
✓	SalesSummaryPanel shows correct total orders and revenue for selected date
✓	No orders for selected date shows a friendly message, not an error
✓	Logout from any dashboard returns to LoginFrame
✓	No SQL code or JDBC imports exist in any UI class
✓	No business logic or calculation exists inside any ActionListener

11. Common Mistakes to Avoid

Mistake	Correct Approach
Writing SQL directly inside an ActionListener	Move all SQL to the DAO class. Call the service from the ActionListener.
Calculating tax inside PlaceOrderPanel	Tax calculation belongs in BillingService.calculateTax() only.
Calling DAO methods directly from a JPanel	Always go through the service layer. JPanel → Service → DAO.
Updating the JTable in memory after adding an item	Always reload the table from the DB using tableModel.setRowCount(0) and re-query.
Printing ex.printStackTrace() to the user	Catch the exception, get ex.getMessage(), and show it in a JLabel or JOptionPane.
Opening a new dashboard without closing the old one	Call dispose() on the current JFrame before opening the next one.
Keeping Admin and Operator in the same dashboard	Use separate AdminDashboard and OperatorDashboard classes for clean role separation.